# Experiment 1 — Caesar Cipher

**Aim**

To write a Python program to implement the **Caesar Cipher** for encryption and decryption.

## Theory

The **Caesar Cipher** is a substitution cipher in which each letter of the plaintext is shifted by a fixed number of positions in the alphabet.

For example, if the key is **3**:

- A → D
- B → E
- C → F
- X → A
- Y → B
- Z → C

The encryption formula is:

**C = (P + K) mod 26**

The decryption formula is:

**P = (C − K) mod 26**

Where:

- **P** = position of plaintext character
- **C** = position of ciphertext character
- **K** = key/shift value

## Algorithm

1. Start the program.
2. Enter the plaintext message.
3. Enter the key/shift value.

4. For encryption, shift every alphabetic character by the key.
5. For decryption, shift every character backward by the key.
6. Display the encrypted and decrypted text.
7. Stop.

## Python Program

```
def caesar_encrypt(text, key):
    result = ""
    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - base + key) % 26 + base)
        else:
            result += char
    return result
def caesar_decrypt(text, key):
    return caesar_encrypt(text, -key)
# Input
text = input("Enter the plaintext: ")
key = int(input("Enter the key: "))
# Encryption
encrypted_text = caesar_encrypt(text, key)
# Decryption
decrypted_text = caesar_decrypt(encrypted_text, key)
print("\nEncrypted Text:", encrypted_text)
print("Decrypted Text:", decrypted_text)
```

## Output:




### Result

Thus, the Caesar Cipher was successfully implemented in Python for encryption and decryption.

# Experiment 2 Playfair Cipher

## Aim

To write a Python program to implement the Playfair Cipher for encryption and decryption.

## Theory

The Playfair Cipher is a symmetric encryption technique that encrypts two letters at a time instead of encrypting individual letters.

It uses a 5 × 5 matrix of alphabets generated from a keyword. Since there are 26 English alphabets but only 25 cells, I and J are generally treated as one letter.

## For encryption:

1. If both letters are in the same row, replace each with the letter to its right.
2. If both letters are in the same column, replace each with the letter below it.
3. If the letters form a rectangle, replace each letter with the letter in the same row but in the other letter's column.
4. If a pair contains the same letter, insert X between them.
5. If the plaintext has an odd number of letters, add X at the end.

## Algorithm

1. Start the program.
2. Enter the keyword.
3. Create a 5 × 5 matrix using the keyword.
4. Remove duplicate letters from the keyword.
5. Add the remaining alphabet letters to complete the matrix.
6. Divide the plaintext into pairs.
7. Insert X between repeated letters in a pair.
8. Encrypt each pair using Playfair rules.
9. Display the encrypted text.
10. Decrypt the ciphertext using the reverse rules.

11. Display the decrypted text.
12. Stop.

## Python Program

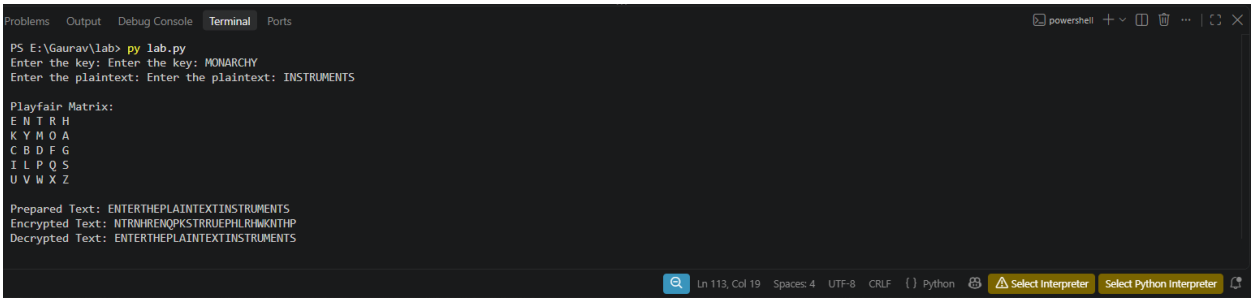
```
def create_matrix(key):
    key = key.upper().replace("J", "I")
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
    result = ""
    # Add key characters
    for char in key:
        if char.isalpha() and char not in result:
            result += char
    # Add remaining alphabet
    for char in alphabet:
        if char not in result:
            result += char
    # Create 5 x 5 matrix
    matrix = [result[i:i + 5] for i in range(0, 25, 5)]
    return matrix
def find_position(matrix, char):
    for row in range(5):
        for col in range(5):
            if matrix[row][col] == char:
                return row, col
def prepare_text(text):
    text = ''.join(char for char in text.upper() if char.isalpha())
```

```
    text = text.replace("J", "I")
    result = ""
    i = 0
    while i < len(text):
        first = text[i]
        if i + 1 < len(text):
            second = text[i + 1]
            if first == second:
                result += first + "X"
                i += 1
            else:
                result += first + second
                i += 2
        else:
            result += first + "X"
            i += 1
    return result
def encrypt(text, matrix):
    result = ""
    for i in range(0, len(text), 2):
        a = text[i]
        b = text[i + 1]
        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)
        if r1 == r2:
```

```
            result += matrix[r1][(c1 + 1) % 5]
            result += matrix[r2][(c2 + 1) % 5]
        elif c1 == c2:
            result += matrix[(r1 + 1) % 5][c1]
            result += matrix[(r2 + 1) % 5][c2]
        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]
    return result
def decrypt(text, matrix):
    result = ""
    for i in range(0, len(text), 2):
        a = text[i]
        b = text[i + 1]
        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)
        if r1 == r2:
            result += matrix[r1][(c1 - 1) % 5]
            result += matrix[r2][(c2 - 1) % 5]
        elif c1 == c2:
            result += matrix[(r1 - 1) % 5][c1]
            result += matrix[(r2 - 1) % 5][c2]
        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]
```

```
    return result
# Main program
key = input("Enter the key: ")
plaintext = input("Enter the plaintext: ")
matrix = create_matrix(key)
print("\nPlayfair Matrix:")
for row in matrix:
    print(" ".join(row))
prepared_text = prepare_text(plaintext)
encrypted_text = encrypt(prepared_text, matrix)
decrypted_text = decrypt(encrypted_text, matrix)
print("\nPrepared Text:", prepared_text)
print("Encrypted Text:", encrypted_text)
print("Decrypted Text:", decrypted_text)
```

## Output:




## Result

Thus, the Playfair Cipher was successfully implemented in Python for encryption and decryption.

# Experiment 3 — Hill Cipher

## Aim

To write a Python program to implement the Hill Cipher for encryption and decryption.

## Theory

The Hill Cipher is a polygraphic substitution cipher based on linear algebra. It encrypts a group of letters together using a square key matrix.

In this experiment, we use a 2 × 2 key matrix.

Each alphabet is represented by a number:

A = 0, B = 1, C = 2, ..., Z = 25

The encryption is performed using:

$C = K \times P \bmod 26$

Where:

- C = Ciphertext matrix
- K = Key matrix
- P = Plaintext matrix
- mod 26 = modulo operation with 26 alphabets

## For decryption:

$P = K^{-1} \times C \bmod 26$

where $K^{-1}$ is the inverse of the key matrix modulo 26.

## Algorithm

1. Start the program.
2. Enter the plaintext.
3. Convert the plaintext into uppercase letters.
4. Convert each letter into its numerical value from 0 to 25.
5. Divide the plaintext into blocks of two letters.

6. Define a 2 × 2 key matrix.
7. Multiply the key matrix with each plaintext block.
8. Apply modulo 26 to obtain ciphertext.
9. Calculate the inverse of the key matrix.
10. Multiply the inverse matrix with the ciphertext.
11. Apply modulo 26 to obtain the original plaintext.
12. Display the encrypted and decrypted text.
13. Stop.

## Python Program

```
import numpy as np

# Convert character to number

def char_to_num(char):

    return ord(char) - ord('A')

# Convert number to character

def num_to_char(num):

    return chr(num % 26 + ord('A'))

# Find modular inverse

def mod_inverse(a, m):

    for x in range(1, m):

        if (a * x) % m == 1:

            return x

    return None

# Find inverse of 2x2 matrix

def matrix_inverse(key):

    a, b = key[0]
```

```
 c, d = key[1]
    determinant = (a * d - b * c) % 26
    inverse_det = mod_inverse(determinant, 26)
    if inverse_det is None:
        raise ValueError("Key matrix is not invertible.")
    inverse = [
        [(d * inverse_det) % 26, (-b * inverse_det) % 26],
        [(-c * inverse_det) % 26, (a * inverse_det) % 26]
    ]
    return inverse
# Encryption
def encrypt(plaintext, key):
    plaintext = plaintext.upper().replace(" ", "")
    # Add X if length is odd
    if len(plaintext) % 2 != 0:
        plaintext += "X"
    ciphertext = ""
    for i in range(0, len(plaintext), 2):
        p1 = char_to_num(plaintext[i])
        p2 = char_to_num(plaintext[i + 1])
        c1 = (key[0][0] * p1 + key[0][1] * p2) % 26
        c2 = (key[1][0] * p1 + key[1][1] * p2) % 26
        ciphertext += num_to_char(c1)
        ciphertext += num_to_char(c2)
    return ciphertext
```

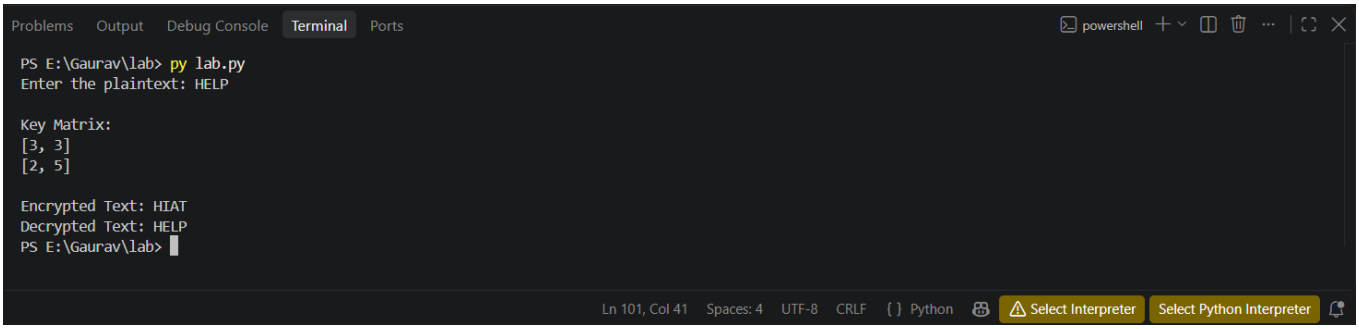
```
# Decryption
def decrypt(ciphertext, key):
    inverse_key = matrix_inverse(key)
    plaintext = ""
    for i in range(0, len(ciphertext), 2):
        c1 = char_to_num(ciphertext[i])
        c2 = char_to_num(ciphertext[i + 1])
        p1 = (inverse_key[0][0] * c1 +
              inverse_key[0][1] * c2) % 26
        p2 = (inverse_key[1][0] * c1 +
              inverse_key[1][1] * c2) % 26
        plaintext += num_to_char(p1)
        plaintext += num_to_char(p2)
    return plaintext
# Main program
plaintext = input("Enter the plaintext: ")
# Key matrix
key = [
    [3, 3],
    [2, 5]
]
print("\nKey Matrix:")
for row in key:
    print(row)
encrypted_text = encrypt(plaintext, key)
```

```
decrypted_text = decrypt(encrypted_text, key)

print("\nEncrypted Text:", encrypted_text)

print("Decrypted Text:", decrypted_text)
```

## Output:




## Result

Thus, the Hill Cipher was successfully implemented in Python for encryption and decryption using a 2 × 2 key matrix.

# Experiment 4 — Vigenère Cipher

## Aim

To write a Python program to implement the Vigenère Cipher for encryption and decryption.

## Theory

The Vigenère Cipher is a polyalphabetic substitution cipher. It uses a keyword to encrypt the plaintext. Unlike the Caesar Cipher, where the same shift is applied to every character, the Vigenère Cipher uses different shifts based on the letters of the key.

The letters are represented as:

A = 0, B = 1, C = 2, ..., Z = 25

Encryption formula:

$$C = (P + K) \bmod 26$$

Decryption formula:

$$P = (C - K) \bmod 26$$

Where:

- P = Plaintext value
- C = Ciphertext value
- K = Key value

## Algorithm

1. Start the program.
2. Enter the plaintext.
3. Enter the keyword.
4. Repeat the keyword to match the length of the plaintext.
5. Convert plaintext and key characters into numerical values.
6. Apply the encryption formula.
7. Generate the ciphertext.

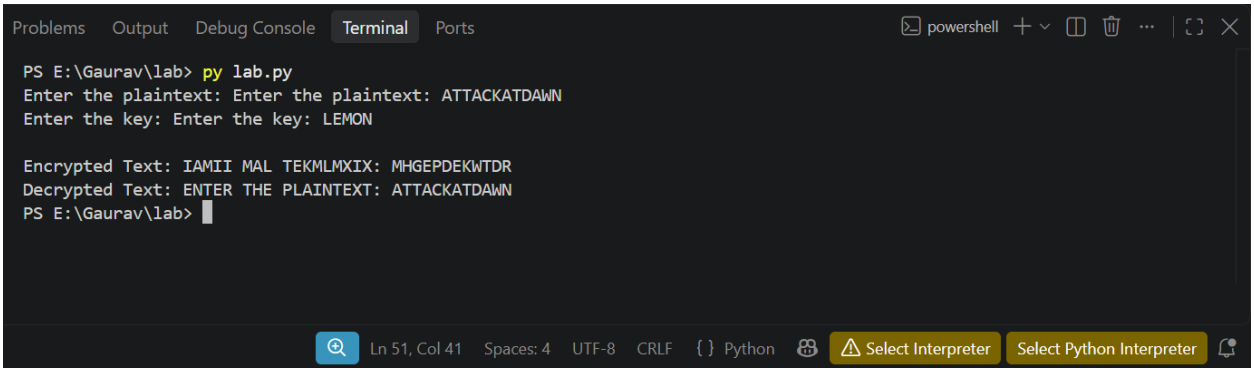
8. Apply the decryption formula to the ciphertext.
9. Display the encrypted and decrypted text.
10. Stop.

## Python Program

```
def encrypt(plaintext, key):
    ciphertext = ""
    key = key.upper()
    key_index = 0
    for char in plaintext:
        if char.isalpha():
            p = ord(char.upper()) - ord('A')
            k = ord(key[key_index % len(key)]) - ord('A')
            c = (p + k) % 26
            ciphertext += chr(c + ord('A'))
            key_index += 1
        else:
            ciphertext += char
    return ciphertext
def decrypt(ciphertext, key):
    plaintext = ""
    key = key.upper()
    key_index = 0
    for char in ciphertext:
        if char.isalpha():
            c = ord(char.upper()) - ord('A')
```

```
            k = ord(key[key_index % len(key)]) - ord('A')
            p = (c - k) % 26
            plaintext += chr(p + ord('A'))
            key_index += 1
        else:
            plaintext += char
    return plaintext
# Main program
plaintext = input("Enter the plaintext: ")
key = input("Enter the key: ")
encrypted_text = encrypt(plaintext, key)
decrypted_text = decrypt(encrypted_text, key)
print("\nEncrypted Text:", encrypted_text)
print("Decrypted Text:", decrypted_text)
```

## Output:




## Result

Thus, the Vigenère Cipher was successfully implemented in Python for encryption and decryption.

# Experiment 5 — Rail Fence Row & Column Transformation

## Aim

To write a Python program to implement **Rail Fence (Row & Column) Transposition Cipher** for encryption and decryption.

## Theory

The **Rail Fence Cipher** is a transposition cipher. It does not change the actual letters of the plaintext; instead, it changes their **positions** according to a fixed number of rails.

For example, with **3 rails**, the plaintext is written in a zig-zag pattern:

W E C R

E R T O E E

A D S

The letters are then read row by row to obtain the ciphertext.

## Algorithm

1. Start the program.
2. Enter the plaintext.
3. Enter the number of rails.
4. Write the plaintext in a zig-zag pattern across the rails.
5. Read the characters row by row to generate ciphertext.
6. For decryption, reconstruct the zig-zag pattern.
7. Place the ciphertext characters into the appropriate positions.
8. Read the characters following the zig-zag pattern to obtain plaintext.
9. Display encrypted and decrypted text.
10. Stop.

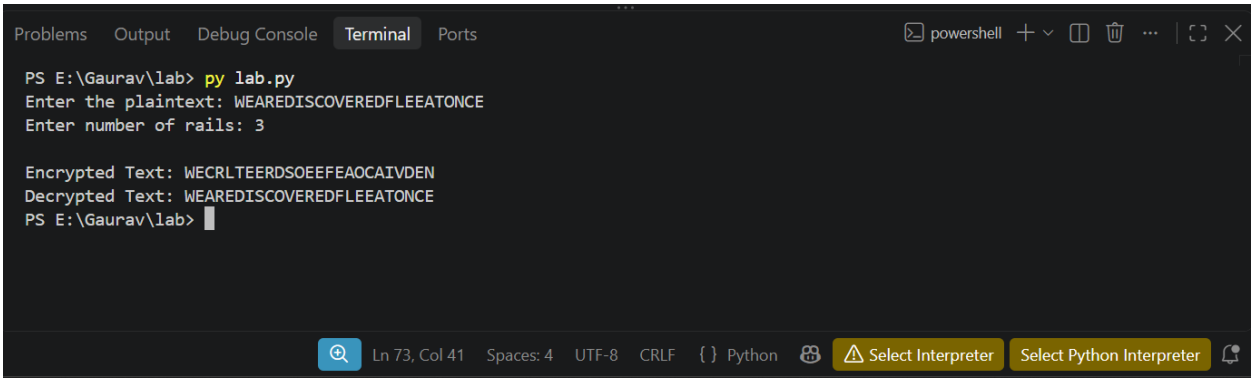
## Python Program

```
def encrypt(text, rails):
```

```
    if rails == 1:
        return text
    fence = [[] for _ in range(rails)]
    row = 0
    direction = 1
    for char in text:
        fence[row].append(char)
        if row == 0:
            direction = 1
        elif row == rails - 1:
            direction = -1
        row += direction
    return ''.join(''.join(row) for row in fence)
def decrypt(ciphertext, rails):
    if rails == 1:
        return ciphertext
    # Find the rail pattern
    pattern = []
    row = 0
    direction = 1
    for _ in ciphertext:
        pattern.append(row)
        if row == 0:
            direction = 1
        elif row == rails - 1:
```

```
            direction = -1
        row += direction
    # Count characters in each rail
    rail_counts = [pattern.count(i) for i in range(rails)]
    # Divide ciphertext among rails
    fence = []
    index = 0
    for count in rail_counts:
        fence.append(list(ciphertext[index:index + count]))
        index += count
    # Reconstruct plaintext
    result = []
    rail_position = [0] * rails
    for rail in pattern:
        result.append(fence[rail][rail_position[rail]])
        rail_position[rail] += 1
    return ''.join(result)
# Main program
plaintext = input("Enter the plaintext: ")
rails = int(input("Enter number of rails: "))
encrypted_text = encrypt(plaintext, rails)
decrypted_text = decrypt(encrypted_text, rails)
print("\nEncrypted Text:", encrypted_text)
print("Decrypted Text:", decrypted_text)
```

## Output:




## Result

Thus, the Rail Fence Cipher was successfully implemented in Python for encryption and decryption using row and column transformation.

# Experiment 6 — DES Algorithm

## Aim

To write a Python program to implement the **Data Encryption Standard (DES)** algorithm for encryption and decryption.

## Theory

**DES (Data Encryption Standard)** is a symmetric-key block cipher. It encrypts data in blocks of **64 bits** using a **56-bit effective key**. The algorithm uses **16 rounds** of processing.

DES uses the **same secret key** for both encryption and decryption.

The main steps of DES are:

1. Initial Permutation
2. Division into left and right halves
3. 16 rounds of Feistel processing
4. Final Permutation

**Note:** Modern systems generally do not use DES for real security because its 56-bit key is too small against modern brute-force attacks. For a practical-file experiment, however, we can demonstrate DES using Python's pycryptodome library.

## Algorithm

1. Start the program.
2. Enter the plaintext.
3. Enter an 8-byte secret key.
4. Convert the plaintext into bytes.
5. Create a DES cipher object.
6. Encrypt the plaintext using DES.
7. Display the encrypted ciphertext.
8. Use the same key to decrypt the ciphertext.
9. Display the original plaintext.

10. Stop.

## Python Program

First install the required library:

```
pip install pycryptodome
```

Then use:

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
# Encryption function
def encrypt(plaintext, key):
    cipher = DES.new(key, DES.MODE_ECB)
    padded_text = pad(plaintext.encode(), DES.block_size)
    encrypted = cipher.encrypt(padded_text)
    return encrypted
# Decryption function
def decrypt(ciphertext, key):
    cipher = DES.new(key, DES.MODE_ECB)
    decrypted = cipher.decrypt(ciphertext)
    return unpad(decrypted, DES.block_size).decode()
# Main program
plaintext = input("Enter the plaintext: ")
key_input = input("Enter 8-character key: ")
# DES requires exactly 8 bytes
if len(key_input) != 8:
    print("Error: Key must contain exactly 8 characters.")
else:
```

```
key = key_input.encode()

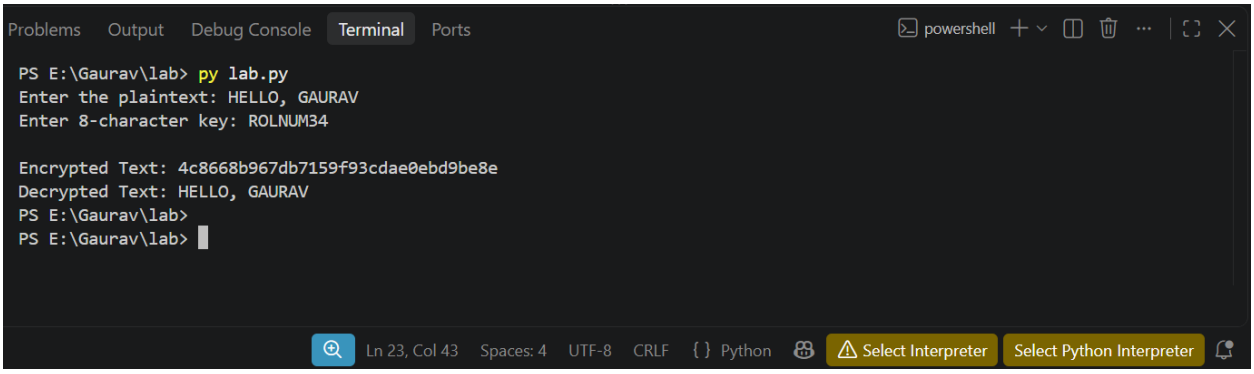
encrypted_text = encrypt(plaintext, key)

decrypted_text = decrypt(encrypted_text, key)

print("\nEncrypted Text:", encrypted_text.hex())

print("Decrypted Text:", decrypted_text)
```

## Output:




## Result

Thus, the DES algorithm was successfully implemented in Python for encryption and decryption using a secret key.

# Experiment 7 — RSA Algorithm

## Aim

To write a Python program to implement the **RSA algorithm** for encryption and decryption.

## Theory

**RSA (Rivest–Shamir–Adleman)** is an **asymmetric-key cryptographic algorithm**. It uses two different keys:

- **Public Key** — used for encryption.
- **Private Key** — used for decryption.

RSA is based on the mathematical difficulty of **factorizing a large number obtained by multiplying two prime numbers**.

For this practical, we use two prime numbers p and q.

The main steps are:

1. Select two prime numbers p and q.
2. Calculate:

$$n = p \times q$$

3. Calculate Euler's Totient:

$$\phi(n) = (p-1)(q-1)$$

4. Select a public exponent e such that:

$$1 < e < \phi(n)$$

and gcd(e, φ(n)) = 1.

5. Calculate the private key d:

$$d \times e \equiv 1 \ \left(\mathrm{mod}\ \phi(n)\right)$$

6. Public key = (e, n)
7. Private key = (d, n)

Encryption:

$$C = M^e \mod n$$

Decryption:

$$M = C^d \mod n$$

**Algorithm**

1. Start the program.
2. Select two prime numbers p and q.
3. Calculate n = p × q.
4. Calculate φ(n).
5. Select the public key e.
6. Calculate the private key d.
7. Display the public and private keys.
8. Convert plaintext characters into numerical values.
9. Encrypt using the public key.
10. Decrypt using the private key.
11. Convert the decrypted values back into characters.
12. Display the encrypted and decrypted text.
13. Stop.

## Python Program

```
from math import gcd

# Function to find the modular inverse

def mod_inverse(e, phi):

    for d in range(1, phi):

        if (e * d) % phi == 1:

            return d

    return None
```

```
# Step 1: Select two prime numbers
p = 61
q = 53
# Step 2: Calculate n
n = p * q
# Step 3: Calculate Euler's Totient
phi = (p - 1) * (q - 1)
# Step 4: Select public key e
e = 17
# Check that e and phi are coprime
if gcd(e, phi) != 1:
    print("Invalid public key.")
else:
    # Step 5: Calculate private key d
    d = mod_inverse(e, phi)
    print("Public Key  :", (e, n))
    print("Private Key :", (d, n))
    # Input plaintext
    plaintext = input("\nEnter the plaintext: ")
    # Encryption
    encrypted = []
    for char in plaintext:
        m = ord(char)
        c = pow(m, e, n)
        encrypted.append(c)
```

```
print("\nEncrypted Text:", encrypted)

# Decryption

decrypted = ""

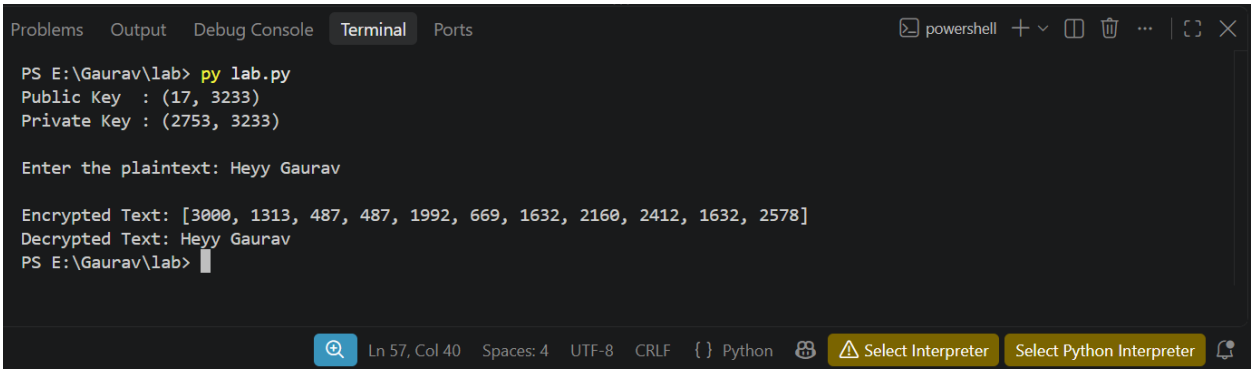
for c in encrypted:

    m = pow(c, d, n)

    decrypted += chr(m)

print("Decrypted Text:", decrypted)
```

## Output:




## Result

Thus, the RSA algorithm was successfully implemented in Python for encryption and decryption using public and private keys.

# Experiment 8 — Diffie-Hellman Key Exchange

## Aim

To write a Python program to implement the **Diffie-Hellman Key Exchange Algorithm**.

## Theory

**Diffie-Hellman Key Exchange** is a cryptographic method that allows two parties to establish a **common secret key** over an insecure communication channel.

The important point is that the secret key itself is **not directly transmitted** between the two parties.

In this algorithm:

- p = a large prime number
- g = primitive root (generator)
- a = private key of Alice
- b = private key of Bob
- A = public key of Alice
- B = public key of Bob

Alice calculates:

$$A = g^a \mod p$$

Bob calculates:

$$B = g^b \mod p$$

Alice calculates the shared secret:

$$K_A = B^a \mod p$$

Bob calculates:

$$K_B = A^b \mod p$$

Both values are equal:

$$K_A = K_B$$

Therefore, both parties obtain the same **shared secret key**.

## Algorithm

1. Start the program.
2. Select a prime number p.
3. Select a generator g.
4. Select Alice's private key a.
5. Select Bob's private key b.
6. Calculate Alice's public key.
7. Calculate Bob's public key.
8. Exchange the public keys.
9. Calculate the shared secret key for Alice.
10. Calculate the shared secret key for Bob.
11. Compare both shared keys.
12. Display the result.
13. Stop.

## Python Program

```
# Diffie-Hellman Key Exchange

# Public values

p = 23

g = 5

# Private keys

a = 6       # Alice's private key

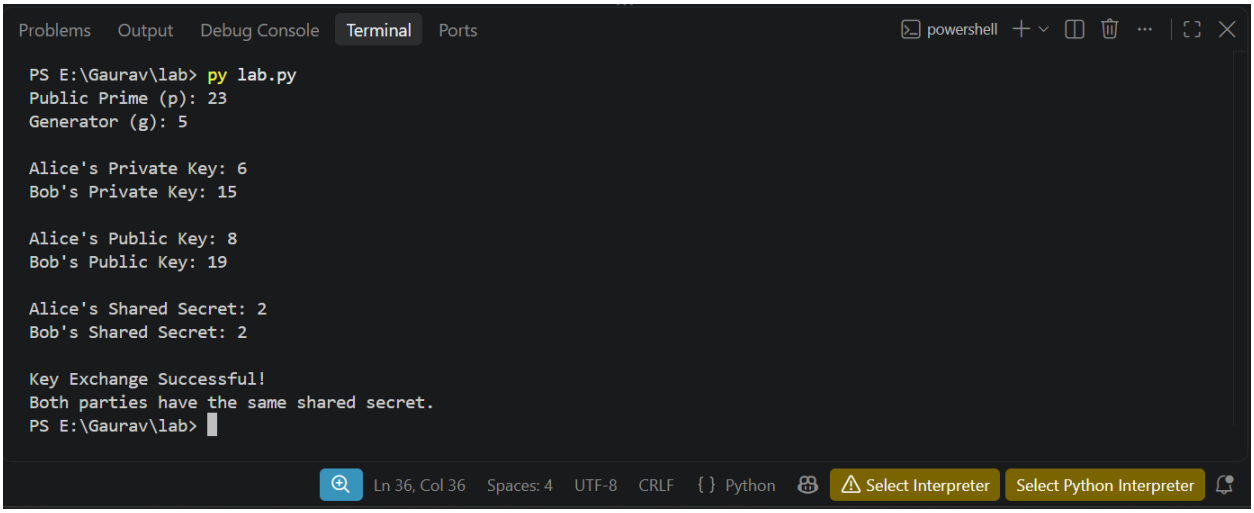
b = 15      # Bob's private key

# Calculate public keys

A = pow(g, a, p)
```

```
B = pow(g, b, p)
print("Public Prime (p):", p)
print("Generator (g):", g)
print("\nAlice's Private Key:", a)
print("Bob's Private Key:", b)
print("\nAlice's Public Key:", A)
print("Bob's Public Key:", B)
# Calculate shared secret keys
alice_secret = pow(B, a, p)
bob_secret = pow(A, b, p)
print("\nAlice's Shared Secret:", alice_secret)
print("Bob's Shared Secret:", bob_secret)
# Check whether both keys are equal
if alice_secret == bob_secret:
    print("\nKey Exchange Successful!")
    print("Both parties have the same shared secret.")
else:
    print("\nKey Exchange Failed!")
```

## Output:




## Result

Thus, the Diffie-Hellman Key Exchange algorithm was successfully implemented in Python, and both parties generated the same shared secret key.

# Experiment 9 — MD5 Algorithm

## Aim

To write a Python program to implement the **MD5 (Message-Digest Algorithm 5)** for generating a hash value of a given message.

## Theory

**MD5** stands for **Message-Digest Algorithm 5**. It is a cryptographic hash function that takes input of any length and produces a fixed-length **128-bit (16-byte) hash value**, usually represented as **32 hexadecimal characters**.

MD5 is a **one-way hash function**. It is not encryption, because the original message cannot normally be obtained by simply "decrypting" the hash.

For example:

Input → HELLO

MD5 → 5d41402abc4b2a76b9719d911017c592

Even a small change in the input produces a very different hash.

**Security note:** MD5 is considered cryptographically broken and should not be used for password storage or modern security applications. It is suitable here for demonstrating the hashing concept.

## Algorithm

1. Start the program.
2. Enter the message.
3. Convert the message into bytes.
4. Apply the MD5 hash function.
5. Generate the 128-bit hash value.
6. Convert the hash into hexadecimal format.
7. Display the MD5 hash.
8. Stop.

## Python Program

```
import hashlib

# Input message

message = input("Enter the message: ")

# Create MD5 hash

md5_hash = hashlib.md5(message.encode())

# Display hash value

print("\nMD5 Hash:", md5_hash.hexdigest())
```

## Output:




## Result

Thus, the MD5 algorithm was successfully implemented in Python to generate the 128-bit hash value of the given message.

# Experiment 10 — SHA-1 Algorithm

## Aim

To write a Python program to implement the **SHA-1 (Secure Hash Algorithm 1)** for generating a hash value of a given message.

## Theory

**SHA-1** stands for **Secure Hash Algorithm 1**. It is a cryptographic hash function that takes an input message and produces a fixed-size **160-bit (20-byte) hash value**.

The SHA-1 hash is normally represented as **40 hexadecimal characters**.

For example:

Input → HELLO

SHA-1 → c65f3f0d4f3e...

SHA-1 is a **hashing algorithm**, not an encryption algorithm. The hash is designed to be a one-way representation of the input.

**Security note:** SHA-1 is no longer considered secure for collision-resistant applications. Modern systems generally use SHA-256 or stronger SHA-2/SHA-3 algorithms. This experiment is for understanding SHA-1 hashing.

## Algorithm

1. Start the program.
2. Enter the message.
3. Convert the message into bytes.
4. Apply the SHA-1 hash function.
5. Generate the 160-bit hash value.
6. Convert the hash into hexadecimal format.
7. Display the SHA-1 hash.
8. Stop.

## Python Program

```
import hashlib

# Input message

message = input("Enter the message: ")

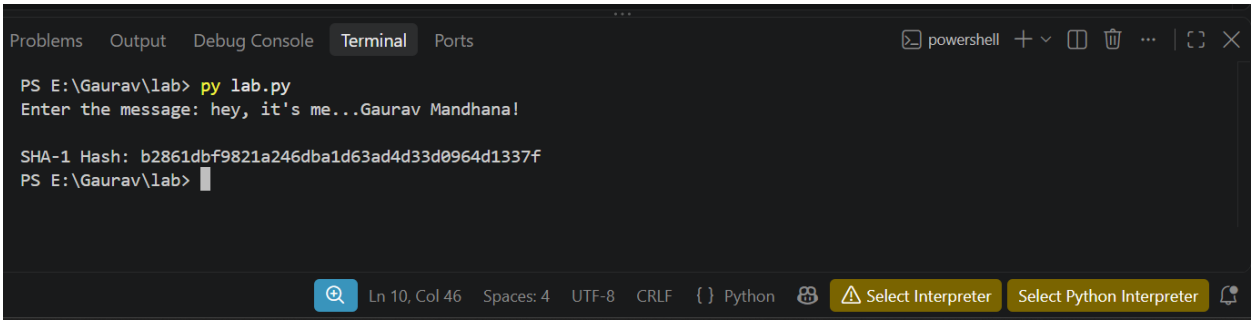
# Create SHA-1 hash

sha1_hash = hashlib.sha1(message.encode())

# Display hash value

print("\nSHA-1 Hash:", sha1_hash.hexdigest())
```

## Output:




## Result

Thus, the SHA-1 algorithm was successfully implemented in Python to generate the 160-bit hash value of the given message.

# Experiment 11 — Digital Signature Standard (DSS)

## Aim

To write a Python program to demonstrate the **Digital Signature Standard (DSS)** for generating and verifying a digital signature.

## Theory

A **Digital Signature** is used to verify the **authenticity and integrity** of a digital message.

The **Digital Signature Standard (DSS)** specifies the use of digital signature techniques. The commonly associated algorithm is **DSA (Digital Signature Algorithm)**.

A digital signature provides:

- **Authentication** — verifies who signed the message.
- **Integrity** — verifies that the message was not changed.
- **Non-repudiation** — provides evidence associated with the signer.

In this practical, Python's cryptography library can be used to generate a DSA private/public key pair, sign a message, and verify the signature.

## Algorithm

1. Start the program.
2. Generate a DSA private key.
3. Generate the corresponding public key.
4. Enter the message.
5. Generate a digital signature using the private key.
6. Display the signature.
7. Verify the signature using the public key.
8. Display whether the signature is valid.
9. Stop.

## Python Program

First install the required library:

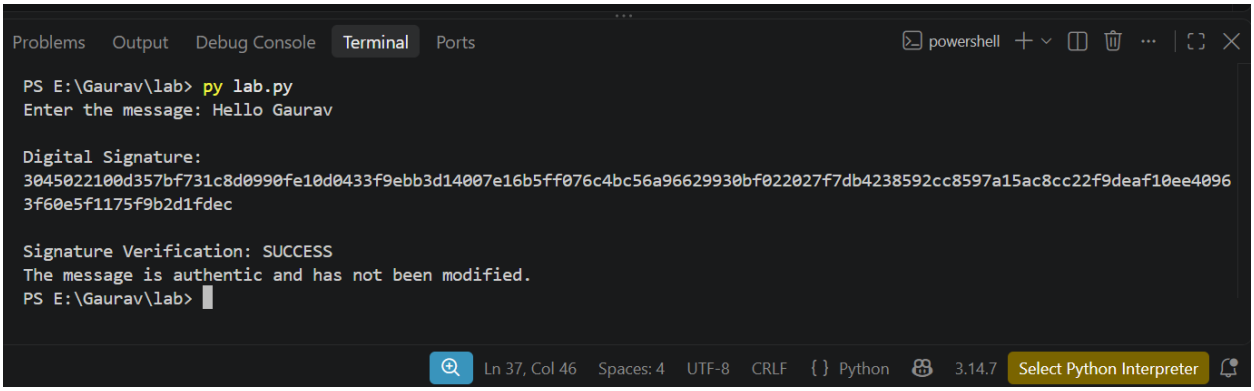
```
pip install cryptography
```

Then run:

```
from cryptography.hazmat.primitives.asymmetric import dsa
from cryptography.hazmat.primitives import hashes
# Generate DSA private key
private_key = dsa.generate_private_key(
    key_size=2048
)
# Generate public key
public_key = private_key.public_key()
# Input message
message = input("Enter the message: ").encode()
# Generate digital signature
signature = private_key.sign(
    message,
    hashes.SHA256()
)
print("\nDigital Signature:")
print(signature.hex())
# Verify signature
try:
    public_key.verify(
        signature,
```

```
        message,
        hashes.SHA256()
    )
    print("\nSignature Verification: SUCCESS")
    print("The message is authentic and has not been modified.")
except Exception:
    print("\nSignature Verification: FAILED")
```

## Output:




## Result

Thus, the Digital Signature Standard was successfully demonstrated in Python by generating and verifying a digital signature using DSA.

# Experiment 12 — Secure Data Storage & Secure Data Transmission

## Aim

To write a Python program to demonstrate **secure data storage and secure data transmission** using encryption.

## Theory

**Secure Data Storage** means protecting data while it is stored so that unauthorized users cannot read it.

**Secure Data Transmission** means protecting data while it is being transferred from one system to another.

In this experiment, we use **Fernet symmetric encryption** from Python's cryptography library. The same secret key is required to encrypt and decrypt the data.

The basic process is:

Original Data

↓

Encrypt

↓

Encrypted Data

↓

Decrypt

↓

Original Data

## Algorithm

1. Start the program.
2. Generate a secure encryption key.
3. Enter the data/message.
4. Encrypt the data using the secret key.

5. Store the encrypted data.
6. Display the encrypted data.
7. Decrypt the encrypted data using the same key.
8. Display the original data.
9. Stop.

## Python Program

```
from cryptography.fernet import Fernet

# Generate secret key

key = Fernet.generate_key()

# Create Fernet object

cipher = Fernet(key)

# Input data

data = input("Enter the data: ")

# Encrypt data

encrypted_data = cipher.encrypt(data.encode())

print("\nSecret Key:", key.decode())

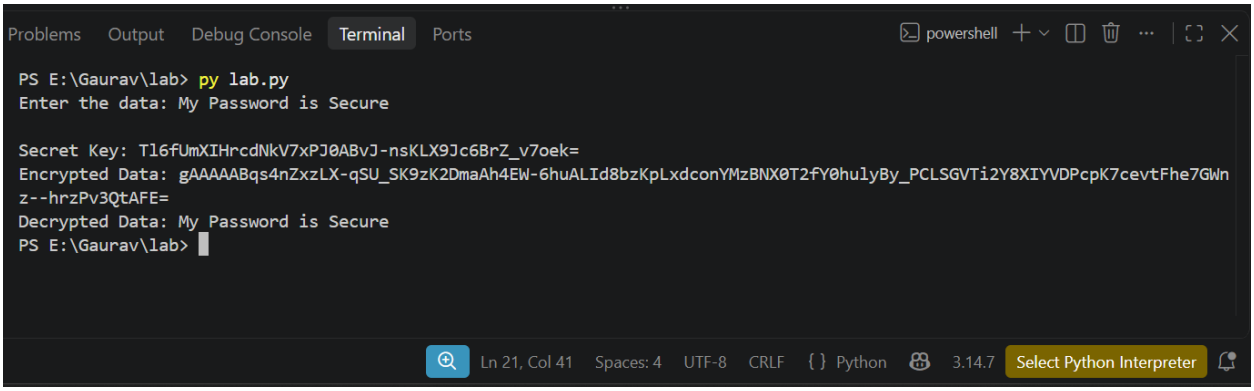
print("Encrypted Data:", encrypted_data.decode())

# Decrypt data

decrypted_data = cipher.decrypt(encrypted_data).decode()

print("Decrypted Data:", decrypted_data)
```

## Output:




## Result

Thus, secure data storage and secure data transmission were successfully demonstrated in Python using Fernet symmetric encryption.